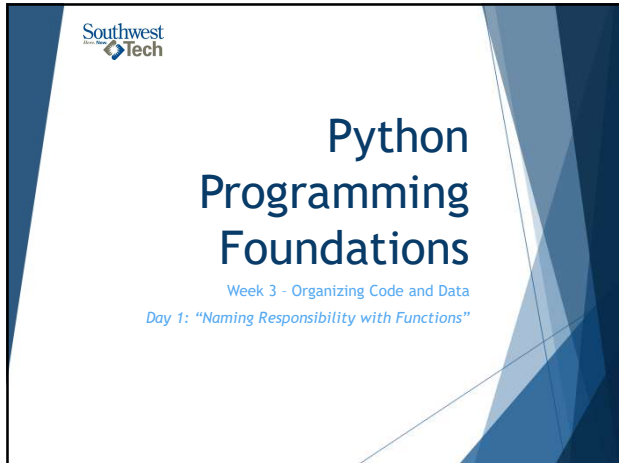
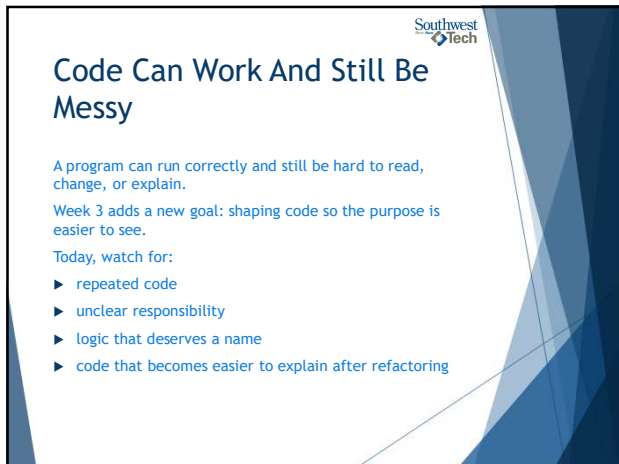
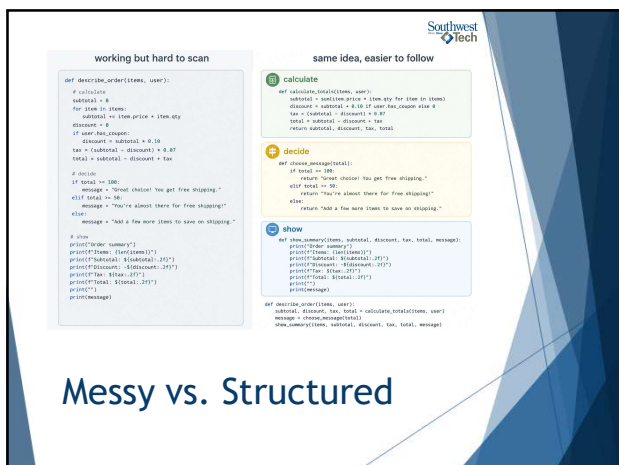




1



2



3



Flow Becomes Shape

Week 2 focused on control flow:

- ▶ conditions choose
- ▶ loops repeat
- ▶ output shows the result

Week 3 asks how that logic should be organized so another person can understand it.

4



Week 2 Concepts Flow Into Functions

Functions organize the skills you already know.

variables

- store values
- keep track
- use in calculations

conditions

- make decisions
- choose a path
- compare values

loops

- repeat steps
- update values
- stop at the right time

function

named responsibility

- Does one clear job from start to finish
- Uses variables, conditions, and loops inside
- Has a name so we can use it again

- returns a result (gives back a value when needed)
- reusable building block (use it in many places without rewriting)

Functions organize what you already know into clear, reusable steps. They don't replace conditions or loops — they put them to work.

Your Week 2 skills → Functions bring them together → Better programs. Easier to build.

“Flow” becomes “Shape”

5

What Counts as Success Today

Repeated or related logic can be named, separated, reused, and explained.

For each function, ask:

What job does it own?	What information does it need?	What result or behavior does it produce?	Where is it called?
-----------------------	--------------------------------	--	---------------------

6

Functions Turn Repetition into Responsibility

The same idea, in four steps.

- 1 Repeated Logic**
The same steps appear in multiple places.
- 2 Named Responsibility**
Give the steps a name and a clear job to do.
- 3 Function Call**
Use the name to run the job when needed.
- 4 Cleaner Program**
Less clutter, easier to read, easier to improve.

Next Step

named responsibility
This function has one job: calculate the total for any list of items.

call the function
total_x = calculate_total(items_x)

cleaner, clearer code
The main program focuses on what happens, not how every detail works.

Functions don't change your logic — they give it a name and a purpose.
Some skills. Better organization.

Today's Success Pattern

7

What We Will Use Today

- ▶ `'def'`
- ▶ function names
- ▶ function calls
- ▶ parameters
- ▶ return values
- ▶ small refactors

These tools help us give code a clear job and reuse that job safely.

8

Functions Toolbox

Simple skills. Powerful results.

def
Start a function definition.
Give it a name.

```
def total(items):  
    for item in items:  
        total = total + item  
    return total
```


Create a named block of work.
Reuse when called.

Function Names
Use clear, specific names that describe the job.

```
total_name = 'calculate_total'  
def total_name(items):  
    # ...  
    return total
```


Good names make code easier to read and use.

Function Calls
Use the function name with parentheses to run the job.

```
total = total_name(items)
```


Calls can appear any time in your program.

Parameters
Put information into a function. Make it flexible.

```
def greet(name):  
    message = 'Hi, ' + name  
    return message  
greet('Maya')
```


Parameters let you reuse the same function.

Return Values
Send a result back to the place that called the function.

```
def square(x):  
    result = x * x  
    return result  
value = square(5)  
print(value)
```


Return values make functions useful and powerful.

Small Refactors
Move repeated logic into a function. Keep improving.

```
def calculate_total(items):  
    total = 0  
    for item in items:  
        total = total + item  
    return total
```


Small changes lead to cleaner, easier programs.

Functions help you organize what you already know. One clear job. Use it again and again.

Today's Toolbox

9

Southwest
Tech

What We Will “Skip” For Now

We will skip recursion, lambdas, decorators, and advanced imports for now.

Today's goal is smaller:

- ▶ find repeated or related logic
- ▶ name one clear responsibility
- ▶ write a function
- ▶ call it correctly



10

Southwest
Tech

Parked for Later
Useful topics.
Not our focus today.

We'll use these. Just not today.
Today we build strong foundations.

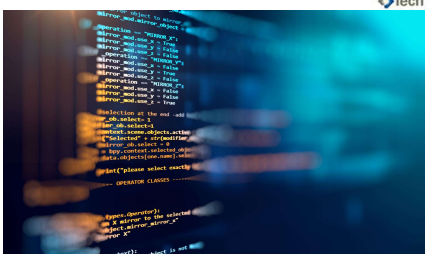
recursion	lambdas	decorators	advanced imports
A function calling itself to solve smaller versions of a problem.	Small anonymous functions used for short, single tasks.	A way to add extra behavior to functions cleanly and predictably.	Tools for organizing and controlling how modules and packages load.
⌚ We'll revisit this later.	⌚ We'll revisit this later.	⌚ We'll revisit this later.	⌚ We'll revisit this later.

Master the basics first. These topics will make even more sense later.

Parked for Later

11

Southwest
Tech



Review Coding Lab

12

Repeated Code Is A Signal

Repeated code often means the program is doing the same job more than once.

That repeated job may deserve a function.

Look for repeated:

- ▶ calculations
- ▶ output formatting
- ▶ validation checks
- ▶ decision patterns

13

Spot the Pattern. Simplify the Code.

When the same job shows up more than once, it's a clue to inspect for a possible function.

First time	Second time
1 # Process scores for math	1 # Process scores for science
2 total = 0	2 total = 0
3 for score in math_scores:	3 for score in science_scores:
4 if score >= 60:	4 if score >= 60:
5 total = total + score	5 total = total + score
6 count = count + 1	6 count = count + 1
7 average_math = total / count	7 average_science = total / count

same job repeated



These blocks do the same job with different data. This repetition is a clue to create a function.



Inspect for a function. Name the job once. Use it many times.

Repeated Code is "Sign"

14

Demo #1

"Repeated Code Before Functions"

15



Demo 1: Repeated Code Before Functions

Watch for repeated logic - Before changing code, identify:

- ▶ which lines repeat
- ▶ what job those lines perform
- ▶ what would be annoying to update later

16



A Function Owns One Clear Job

A function is a named piece of code with a clear responsibility.

Good function names usually describe the job:

- ▶ `'calculate_total'`
- ▶ `'get_status'`
- ▶ `'show_result'`

If the name is hard to choose... the job may not be clear yet.

17



One Function. One Job.

Clear input. Clear output. Clear responsibility.

INPUT

amount: 100.00
tax_rate: 0.07

JOB:

`calculate_total`

`calculate total with tax`



This function has one job and does it well.

OUTPUT

total_with_tax: 107.00

★ One function. One responsibility.
Easy to understand. Easy to test. Easy to reuse.

Functions have only ONE job

18

Southwest
Tech

Function Shape

Basic function shape:

```
1 def function_name(parameter):  
2     result = parameter + 1  
3     return result  
4
```

The definition creates the function. The call runs it.

19

Southwest
Tech

Definition vs Call

Define it once. Call it when you need it.

DEFINITION

```
def calculate_total(price):
```

What it means:
Create a named set of steps that does the job.

This is where the job is defined.
It runs only when called.

CALL

```
calculate_total(12.00)
```

What it means:
Run the named job now with the given value.

This starts the job.
It uses the steps from the definition.

Define once. Call many times. Keep your code clear and simple.

Function Signature (“Shape”)

20

Demo #2

“Refactor Into a Function”

21



Demo 2: What to Watch For

Watch what changes after the repeated logic becomes a function.

Trace:

- ▶ the function definition
- ▶ the parameter going in
- ▶ the return value coming out
- ▶ the function call being reused

22



From Repetition to Clarity

Same job. Less code. Easier to read.

BEFORE

Repeated tax calculation

```

# Order 1
subtotal = order1_subtotal
tax = subtotal * TAX_RATE
total1 = subtotal + tax

# Order 2
subtotal = order2_subtotal
tax = subtotal * TAX_RATE
total2 = subtotal + tax

# Order 3
subtotal = order3_subtotal
tax = subtotal * TAX_RATE
total3 = subtotal + tax

```

Same job. Written 3 times.

Harder to read. Harder to update. Change the logic in 3 places.

AFTER

One function, three calls

```

def calculate_total(subtotal):
    1. tax = subtotal * TAX_RATE
    2. total = subtotal + tax
    3. return total

```

Clear responsibility: calculate total with tax.

Use it anywhere:

```

total1 = calculate_total(order1_subtotal)
total2 = calculate_total(order2_subtotal)
total3 = calculate_total(order3_subtotal)

```

Called 3 times. No repetition.

Cleaner and easier to maintain. Change the logic in 1 place.

One clear function. Many uses. Less repetition. More clarity.

Refactoring

23



Values Come In, Results Come Out

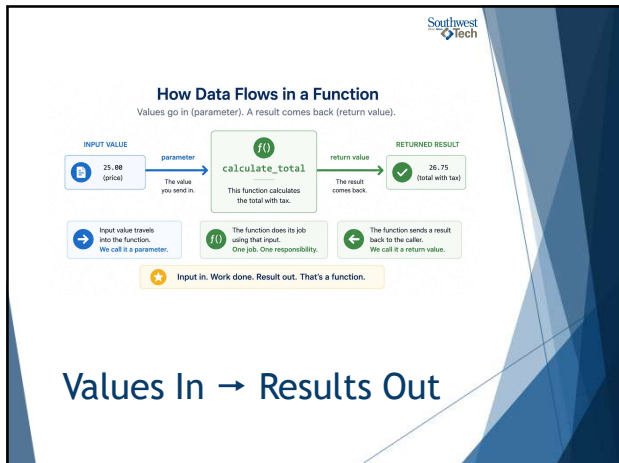
Parameters are information the function receives.

Return values are information the function sends back.

For the beginning mental model:

input value -> function job -> returned result

24



25



26

Southwest Tech

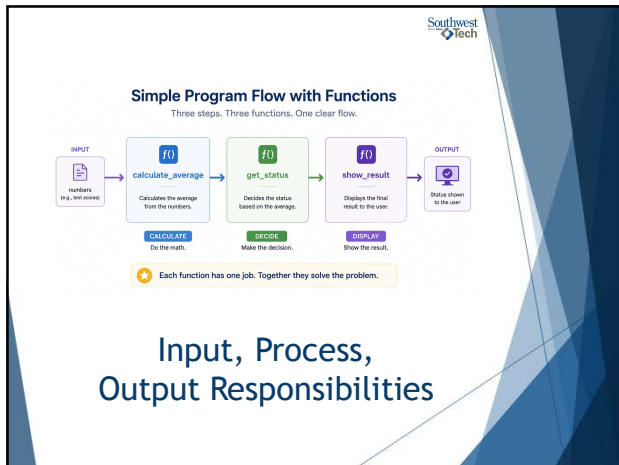
Demo 3: Input, Process, Output Responsibilities

Watch how the program separates jobs:

- ▶ calculate the average
- ▶ decide the status
- ▶ show the result

Each function has a responsibility that can be named and explained.

27



28

Southwest
Tech

Common Failure: More Functions Is Not Better

More functions does not automatically mean better structure.

A useful function should have:

- ▶ a clear job
- ▶ a meaningful name
- ▶ a reason to exist
- ▶ code that is easier to explain because of it

*Tiny random fragments
can make code harder,
not easier.*

29

Southwest
Tech

INTENTIONAL

Clear functions with purpose

```

graph TD
    F1[f0  
calculate_total  
Calculate the total  
with tax.] --> F2[f0  
get_status  
Determine the status  
based on the total.]
    F2 --> F3[f0  
show_result  
Display the final result  
to the user.]
  
```

✔ Clear responsibility. Easy to read. Easy to change.

TOO FRAGMENTED

Many small pieces with no clear purpose

```

total = 0
tax = total * r
print("OK")
if total > 50:
    status = "High"
else:
    status = "Low"
print(status)
if discount > 0:
    total -= discount
    print("Here")
  
```

✗ Harder to read. Harder to change. Easy to miss things.

More Functions are **Not** Better

30

Southwest
Tech

Common Failure: Vague Function Names

Names should help the reader understand the job.

Weak names:

- ▶ `'do_it'`
- ▶ `'thing'`
- ▶ `'process'`

Clearer names:

- ▶ `'calculate_total'`
- ▶ `'get_status'`
- ▶ `'show_result'`

31

Southwest
Tech

Assignment #4

For Assignment 4, build or revise one small program using functions.

A good first version should include:

- ▶ at least two functions
- ▶ meaningful function names
- ▶ correct function calls
- ▶ one parameter or return value if introduced
- ▶ behavior you can explain

32

Southwest
Tech

From Small Program to Named Functions

Small Program (All in One)

```
# get values
price = float(input("Price: "))
tax_rate = 8.87

# calculate total
tax = price * tax_rate
total = price + tax

# show result
print("Total with tax:", total)
```

It works, but the steps are mixed together. Harder to read and reuse.

→

Same Program, Two Named Functions

```
def calculate_total:
    Calculate the total with tax.
    tax = price * tax_rate
    total = price + tax
    return total

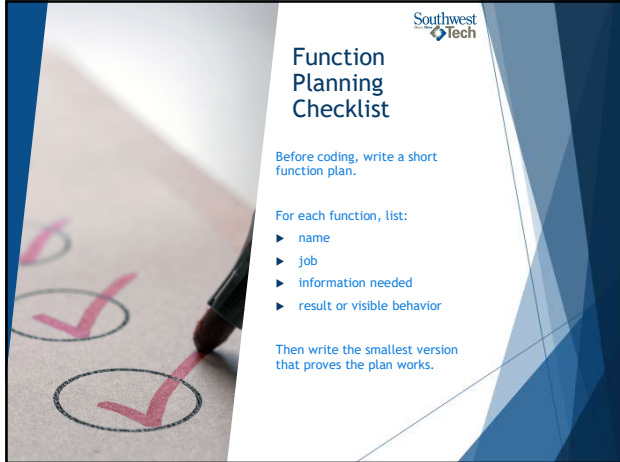
def show_result:
    Display the total to the user.
    print("Total with tax:", total)
```

Each function has one clear job. Easier to read, test, and reuse.

✓ Name the jobs before adding more code.

Assignment #4

33



Function Planning Checklist

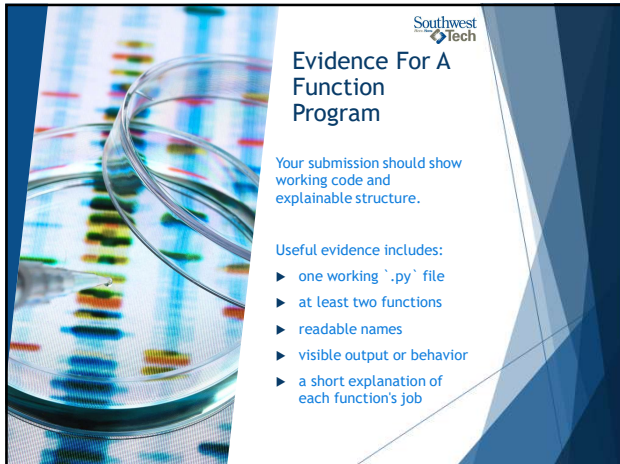
Before coding, write a short function plan.

For each function, list:

- ▶ name
- ▶ job
- ▶ information needed
- ▶ result or visible behavior

Then write the smallest version that proves the plan works.

34



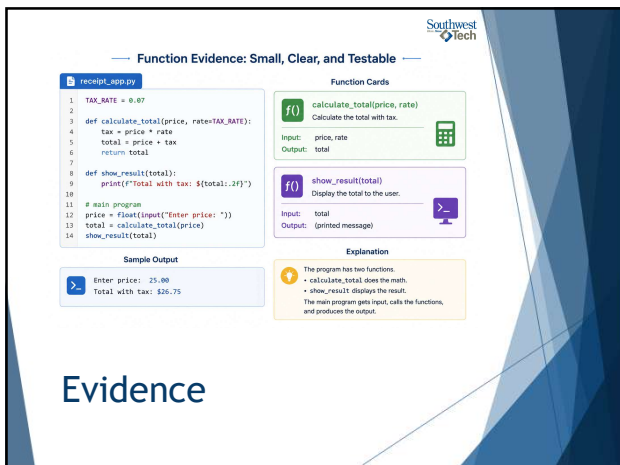
Evidence For A Function Program

Your submission should show working code and explainable structure.

Useful evidence includes:

- ▶ one working `.py` file
- ▶ at least two functions
- ▶ readable names
- ▶ visible output or behavior
- ▶ a short explanation of each function's job

35



Function Evidence: Small, Clear, and Testable

```

1 TAX_RATE = 0.07
2
3 def calculate_total(price, rate=TAX_RATE):
4     tax = price * rate
5     total = price + tax
6     return total
7
8 def show_result(total):
9     print(f"Total with tax: ${total:.2f}")
10
11 # main program
12 price = float(input("Enter price: "))
13 total = calculate_total(price)
14 show_result(total)

```

Sample Output

```

Enter price: 25.00
Total with tax: $26.75

```

Function Cards

f() calculate_total(price, rate)
Calculate the total with tax.
Input: price, rate
Output: total

f() show_result(total)
Display the total to the user.
Input: total
Output: (printed message)

Explanation

The program has two functions:

- calculate_total does the math.
- show_result displays the result.

The main program gets input, calls the functions, and produces the output.

Evidence

36



37
